

Nachdenkzettel: Collections

1. ArrayList oder LinkedList – wann nehmen Sie was?

Operation	ArrayList	LinkedList
Hinzufügen von Elementen	Braucht mehr Zeit	Braucht weniger Zeit
Zugriff auf Element	Braucht weniger Zeit	Benötigt mehr Zeit
Element entfernen	Benötigt mehr Zeit	Braucht weniger Zeit
Elemente iterieren	Braucht weniger Zeit	Braucht mehr Zeit
Element suchen	Braucht weniger Zeit	Benötigt mehr Zeit

2. Interpretieren Sie die Benchmarkdaten von: <http://java.dzone.com/articles/java-collection-performance>. Fällt etwas auf?



Die TreeList braucht länger.
Die HashList hat die schnellere Performance, wenn man in ein Array konvertiert.
CopyOnWriteArray ist langsamer in der Datenmodifikation

3. Wieso ist CopyOnWriteArrayList scheinbar so langsam?

Das CopyOnWriteArray ist synchronisiert. Es ist also eine Thread-Safe-Variante einer ArrayList. Und die Varianten können implimentiert werden indem eine Kopie des Arrays gemacht wird dadurch scheint es einfach langsamer zu sein als die ArrayList.

4. Wie erzeugen Sie eine thread-safe Collection (die sicher bei Nebenläufigkeit ist) (WAS?? die ArrayLists, Linkedlists, Maps etc. sind NICHT sicher bei multithreading??? Wer macht denn so einen Mist???)

By-Default haben vielr Collectiontypes keine interne Synchronisation.
Eine Collection kann Threadsafe gemacht werden,
indem Collection.synchronizedCollection() verwendet wird.

5. Achtung Falle!

```
List<Integer> list = new ArrayList<Integer>;
```

```
Iterator<Integer> itr = list.iterator();
while(itr.hasNext()) {
    int i = itr.next();
    if (i > 5) { // filter all ints bigger than 5
        list.remove();
    }
}
```

Falls es nicht klickt: einfach ausprobieren...

Macht das Verhalten von Java hier Sinn?

Gibt es etwas ähnliches bei Datenbanken? (Stichwort: Cursor. Ist der ähnlich zu Iterator?)

6. Nochmal Achtung Falle: What is the difference between get() and remove() with respect to Garbage Collection?

get() – Methode: Zugriff auf ein einzelnes Element.
Die Collection wird auf keinsten Weise verändert.
remove() – Methode: Element von Collection wird gelöscht.

7. Ihr neuer Laptop hat jetzt 8 cores! Ihr Code für die Verarbeitung der Elemente einer Collection sieht so aus:

```
Iterator<Integer> itr = list.iterator();
while(itr.hasNext()) {
    int i = itr.next();
    //do something with i...
}
```

War der Laptop eine gute Investition?

Für die Mutigen: mal nach map/reduce googeln!

Nein: Der Laptop verwendet nur einer der 8 Cores.